

Clase 02. Herencia

• Agenda

- ✓ Revisión general de programa del curso
- ✓ Consultas y dudas sobre la clase anterior
- ✓ Concepto general de Herencia
- ✓ Superclases y subclases
- ✓ Relación entre superclases y subclases
- ✓ Miembros protegidos
- ✓ Constructores en subclases
- ✓ SobreEscritura



Concepto de herencia

Según (Deitel, 2015), la herencia “es una forma de reutilización de software en la que se crea una nueva clase absorbiendo los miembros de una clase existente, y se mejoran con nuevas capacidades, o con modificaciones en las capacidades ya existentes.”

La herencia es una forma de expresar similitud entre clases.



Muchas veces podemos observar que tenemos clases con algunas características similares y unas pocas diferencias.

.

Igualmente podemos ver claramente cuando un elemento es de alguna manera un "especialización de otro".



Esto nos da una idea de la relación que podemos tener entre clases, donde algunas son una especialización de otras.

Estas "relaciones" representan lo que conocemos como HERENCIA.

fidélitas Universidad


Ideas iniciales
1
Existen clases con algunas características similares y unas pocas diferencias
2
Así, una clase es de alguna manera una "especialización de otra"
3
Las clases se pueden relacionar entre ellas cuando presenten semejanzas
4
La herencia es una forma de expresar semejanzas entre clases.

- Para comprender mejor…



Se tiene un clase “Base” que es la clase más genérica o superior de algún tipo de elementos.



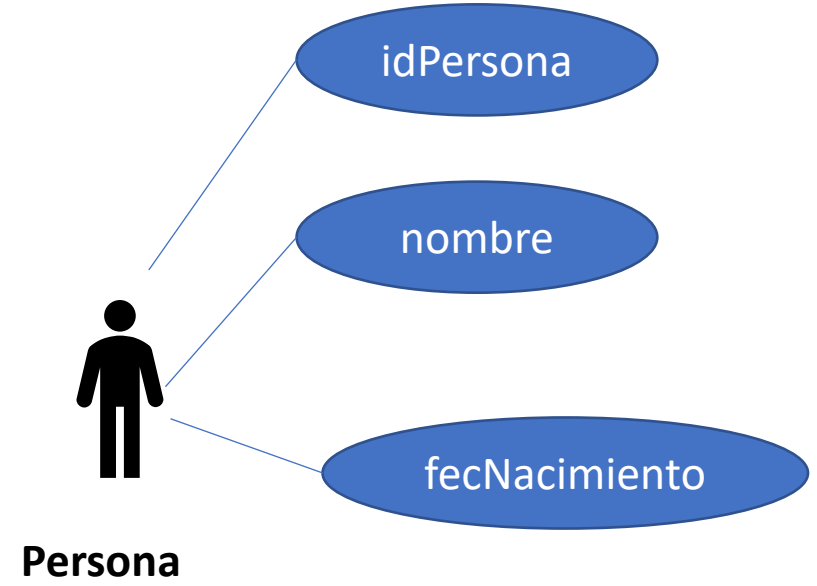
De la clase Persona pueden derivarse “Empleado”, “Estudiante”, “Proveedor”, “Docente”, “Gerente”.



Por ejemplo, la clase “Persona”, es la más genérica que podamos tener en un sistema de manejo de personal.



Otras relaciones… pensemos en “Vehiculo”, se podría derivar “Bus”, “Bicicleta”, “Motocicleta”, “Barco”, “Avion”.



Por ejemplo…

Veamos los atributos elementales de Persona.

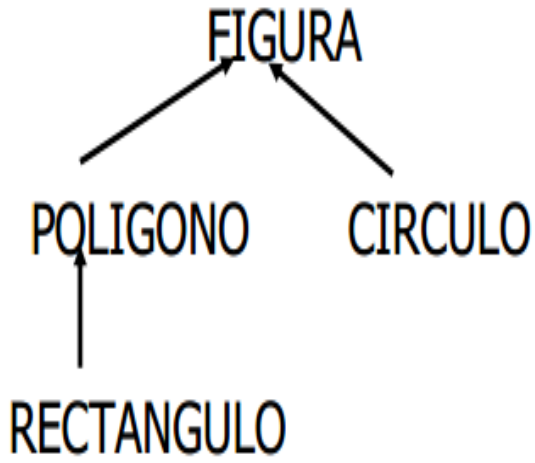
- idPersona, nombre y fecNacimiento son sus atributos esenciales. La clase Empleado tendría sus propios atributos.
- Decimos entonces que Persona es una clase base o superclase y Empleado es una subclase.

fidélitas Universidad

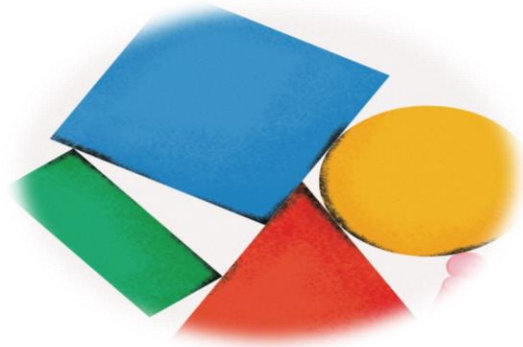
Clases y subclases

La herencia organiza las clases en una estructura jerárquica de clases.





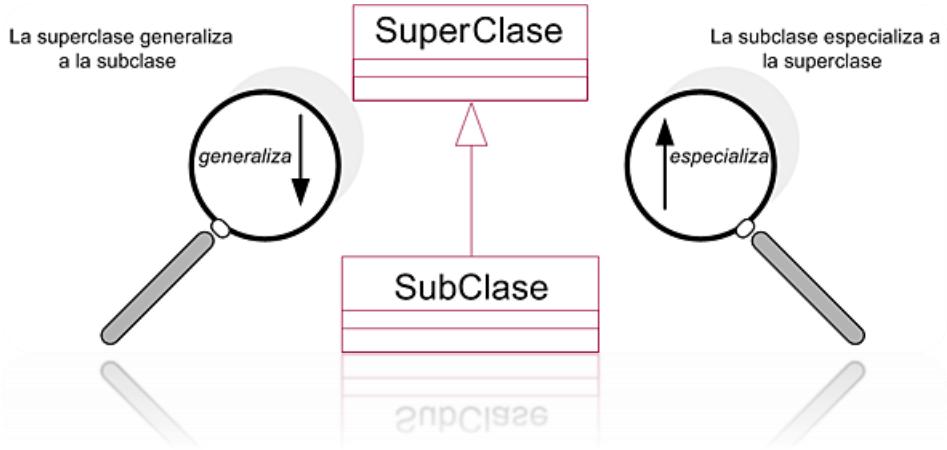






• Clase y subclase

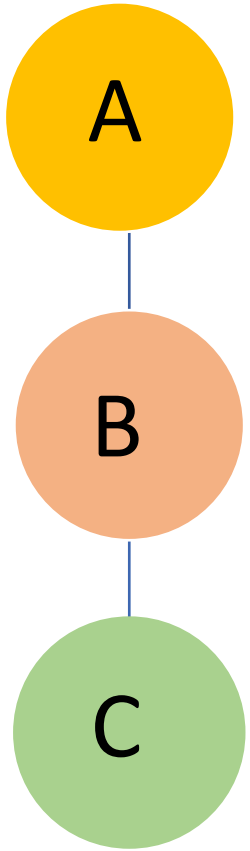
- Si una clase B hereda de otra clase A entonces:
  - B incorpora la estructura (atributos) y comportamiento (métodos) de la clase A
  - B puede incluir adaptaciones:
    - Añadir nuevos atributos
    - Añadir nuevos métodos
    - Redefinir métodos





• Jerarquía

- B hereda de A
  - A es la superclase y B la subclase
- C hereda de B y A
  - B y C son subclases de A
  - B es un descendiente directo de A
  - C es un descendiente indirecto de A



fidélitas
Universidad

Ejemplo

Creemos una clase Persona:



La clase Cliente hereda de Persona

Vendedor igual hereda de Persona



Si creamos una clase Vehículo



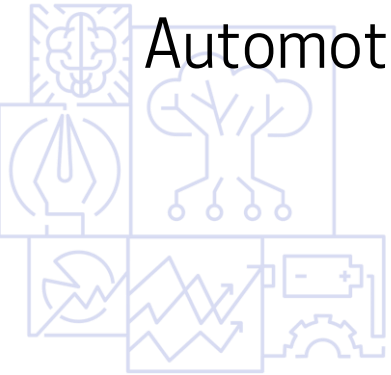
¿Qué serán las clases

Terrestre, Aéreo, Naval



Automotor, Bicicleta, Motocicleta, Tráiler, Autobús?





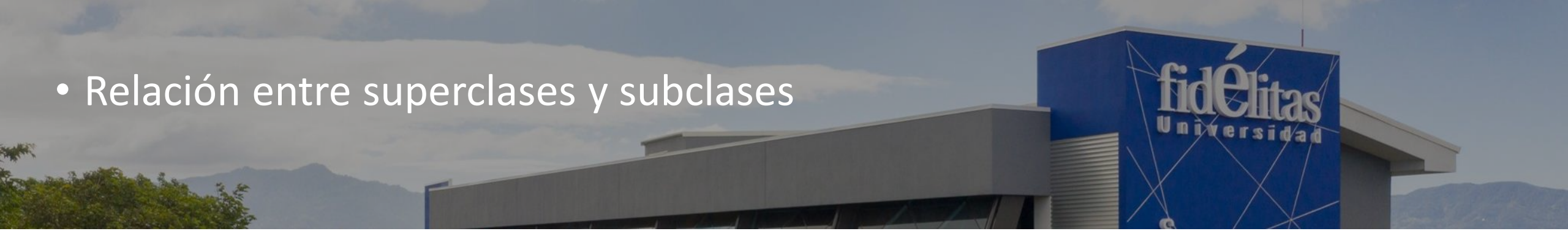
fidélitas Universidad



- Clases y subclases

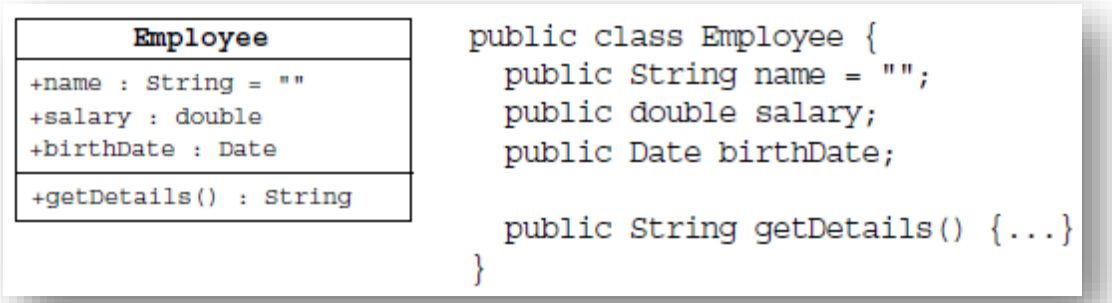
- ✓ Dibuje las jerarquías de clases del ejemplo anterior.
- ✓ Indique cuáles son las clases base, cuáles son las subclases.



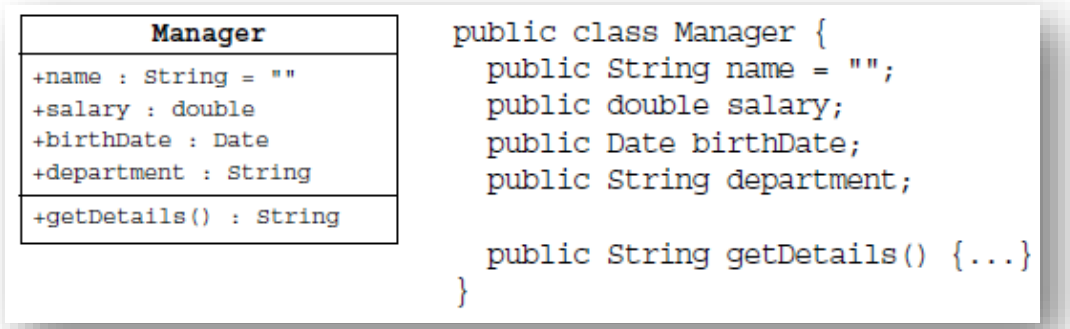


- Relación entre superclases y subclases

Tenemos la clase Empleado



Y la clase Administrador



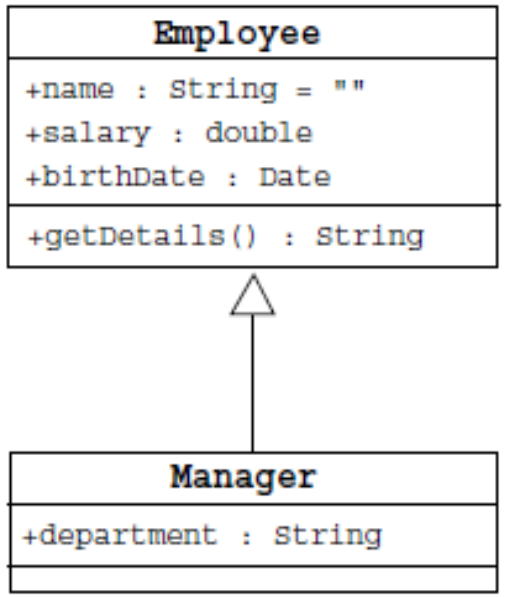
Evidentemente tienen una relación



fidélitas Universidad



- Diagrama de clases para Empleado y Administrador utilizando herencia

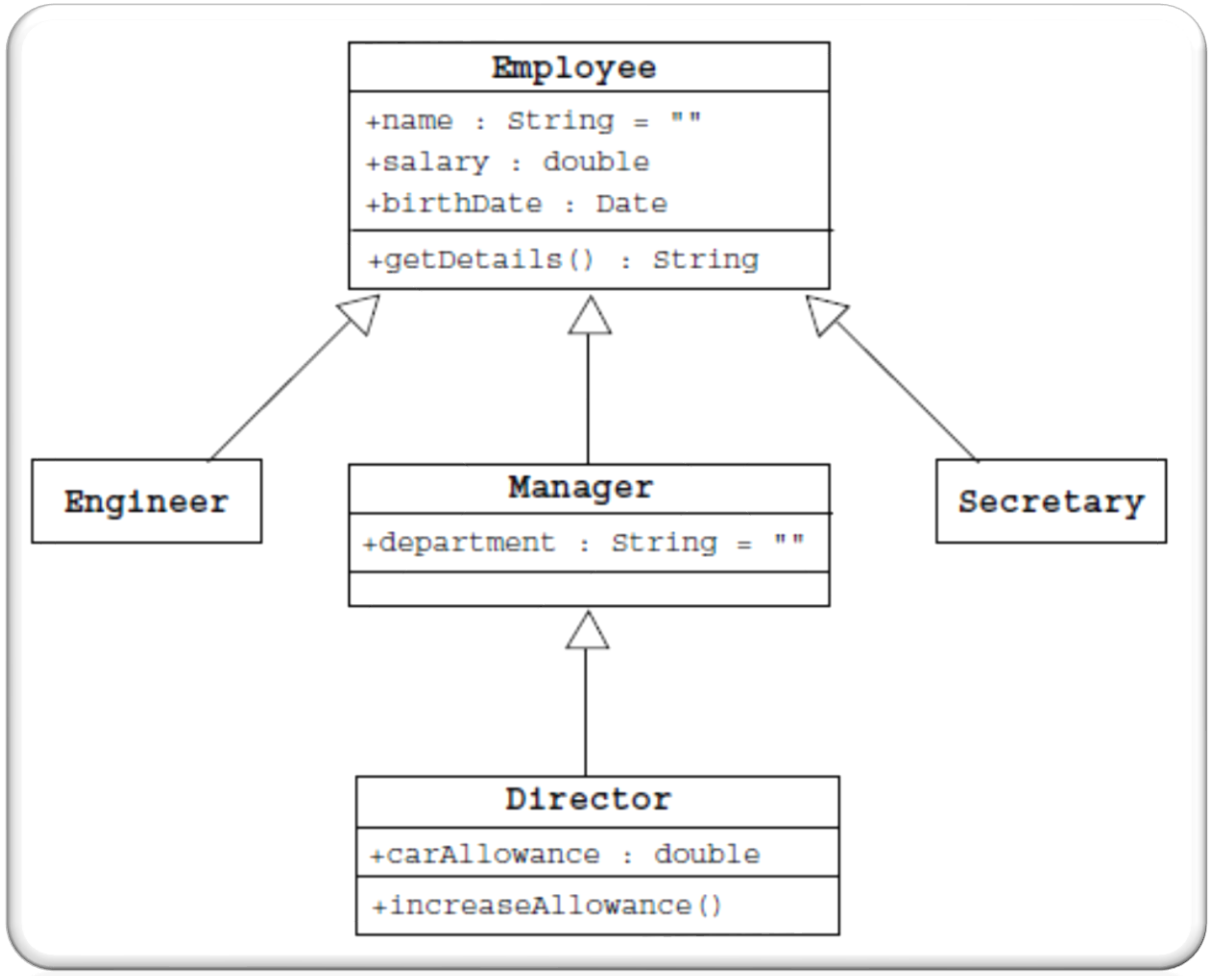



```
public class Employee {
  public String name = "";
  public double salary;
  public Date birthDate;

  public String getDetails() {...}
}
```

```
public class Manager extends Employee {
  public String department;
}
```

- Una superclase puede heredar a varias subclases.





- Miembros protegidos

Los elementos privados no son visibles en las subclases, para ello se utiliza el modificador "protected".

protected, es un modificador intermedio entre public y private. Permite el acceso de los miembros en las subclases.



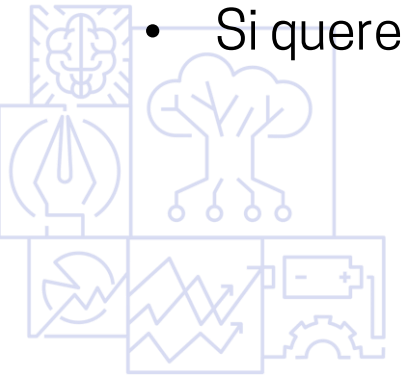
Modificador	Misma clase	Mismo paquete	Subclases	Universo
private	Si			
default	Si	Si		
protected	Si	Si	Si	
public	Si	Si	Si	Si

fidélitas Universidad



• Constructores en subclases

- Los constructores NO se heredan
- Si se define un constructor en una subclase, en principio se debería invocar al constructor de la superclase y en constructor de la superclase llamará al constructor de su superclase y así sucesivamente.
- Para invocar a un elemento de la superclase se utiliza la palabra clase super.
- Si queremos invocar a un constructor lo hacemos con super().





fidélitas
Universidad



- Sobreescritura de métodos

Una subclase puede modificar el comportamiento heredado de una clase padre.



Una subclase puede crear un método con diferente funcionalidad que el método padre, pero con la misma firma (nombre, tipo de retorno y lista de argumentos).

fidélitas
Universidad

- Ejemplo de sobreescritura

```
1   public class Employee {
2     protected String name;
3     protected double salary;
4     protected Date birthDate;
5
6     public String getDetails() {
7       return "Name: " + name + "\n" +
8              "Salary: " + salary;
9     }
10  }
```

```
1   public class Manager extends Employee {
2     protected String department;
3
4     public String getDetails() {
5       return "Name: " + name + "\n" +
6              "Salary: " + salary + "\n" +
7              "Manager of: " + department;
8     }
9   }
```

fidélitas Universidad

- Un método sobreescrito no puede tener un acceso menor

```
1   public class Parent {
2     public void doSomething() {}
3   }

1   public class Child extends Parent {
2     private void doSomething() {} // illegal
3   }

1   public class UseBoth {
2     public void doOtherThing() {
3       Parent p1 = new Parent();
4       Parent p2 = new Child();
5       p1.doSomething();
6       p2.doSomething();
7     }
8   }
```

fidélitas Universidad

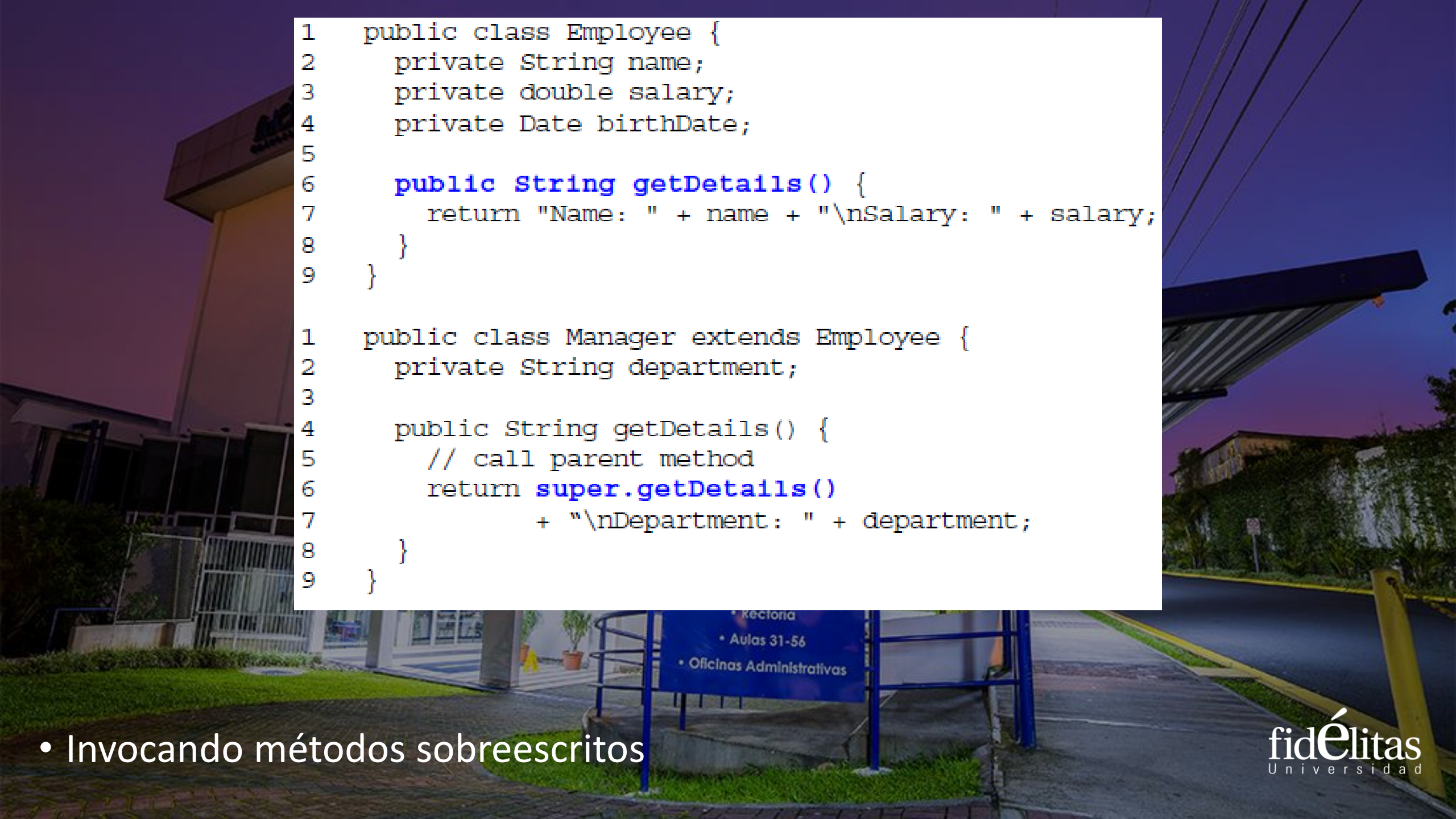


• Invocando métodos sobreescritos

- Una subclase puede llamar al método sobre escrito con la palabra clase super:
  - La palabra clase super es utilizada para referirse al método de una clase superior.
  - Igualmente, es utilizada para referirse a miembros de la superclase: atributos y métodos.
  - El comportamiento invocado no tiene que estar en la superclase, debe estar básicamente arriba en la jerarquía.



fidélitas
Universidad




```java
public class Employee {
  private String name;
  private double salary;
  private Date birthDate;

  public String getDetails() {
    return "Name: " + name + "\nSalary: " + salary;
  }
}
```

```java
public class Manager extends Employee {
  private String department;

  public String getDetails() {
    // call parent method
    return super.getDetails()
          + "\nDepartment: " + department;
  }
}
```

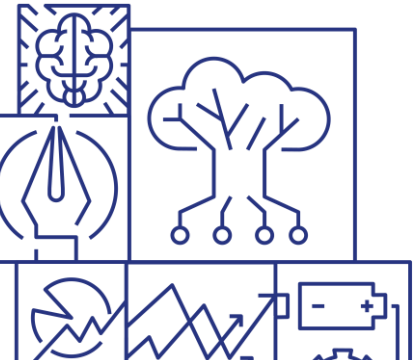
- Invocando métodos sobreescritos

fidélitas Universidad

Práctica



- Cree una jerarquía de clases que contenga al menos 8 Clases diferentes con al menos 3 niveles jerárquicos (Herencia)
- Debe crear atributos en todas la clases, métodos igualmente.
- Se debe hacer uso de sobreescritura, invocar al constructor de la superclase y demás.
- El diseño de clases debe ser una estructura de la vida real, un ejemplo aplicado desde la perspectiva de cada estudiante.



fidélitas Universidad



- Referencias

- Deitel, H. (2015). Java. Estructuras de datos y algoritmos fundamentales. México: Editorial Digital Tecnológico de Monterrey.

fidélitas
Universidad



¡Hasta la próxima!
Nos vemos

Tema de la siguiente clase:
Polimorfismo

fidélitas
Universidad